

一、实验目的

本实验选用 Apache JMeter 对 Spring Boot REST 示例项目 gs-rest-service 进行系统性能测试，掌握脚本录制与增强（事务、参数化、检查点）、多场景压测配置及瓶颈分析方法。

二、被测系统

项目：gs-rest-service (Spring 官方指南)

GitHub: <https://github.com/spring-guides/gs-rest-service>

接口：GET <http://127.0.0.1:8080/greeting?name={name}>

返回：{"id":1, "content":"Hello, {name}!"}

三、脚本增强说明

1. 事务 (Transaction Controller)

使用 Transaction Controller 「TX_Greeting_Complete」将 GET /greeting 请求及断言封装为完整业务事务，便于统计端到端响应时间与吞吐量。

2. 参数化 (CSV Data Set Config)

通过 jmeter/data/names.csv 为 name 参数提供 16 组不同用户名，循环复用，模拟多用户访问。

3. 检查点 (Assertions)

- 响应内容检查点：响应体包含 "Hello"
- HTTP 状态码检查点：响应码为 200
- 响应时间检查点：Duration Assertion (场景1 ≤500ms, 场景2 ≤2000ms)

四、测试场景设计

设置两个用户场景，并监测响应时间、吞吐量、错误率及服务器 CPU/内存指标。

场景	场景1 - 性能测试	场景2 - 压力测试
测试类型	性能测试（基准负载）	压力测试（高并发）
并发用户数	15	50
Ramp-Up (秒)	10	5
循环次数	20	30
总请求数	300	1500
思考时间	100-300ms 随机	50-150ms 随机

监测指标

- JMeter：响应时间、吞吐量 (TPS)、错误率、连接时间、Latency、Apdex
- 服务器：JVM 进程 CPU 使用率、工作集内存 (MB)
- 时间分解：连接时间、网络/等待时间、下载时间、总耗时

五、场景1测试结果

样本数: 300 错误率: 0.0% 吞吐量: 22.48 TPS

响应时间

均值1.64 / P95 3.0 / 最大25.0 ms

时间分解 (ms)

- 连接时间(Connect): 均值0.1 / P95 0.0 / 最大18.0 ms
- 网络/等待时间(Latency-Connect): 均值1.51 / P95 3.0 / 最大7.0 ms
- 下载时间(Elapsed-Latency): 均值0.03 / P95 0.0 / 最大1.0 ms

服务器资源

- CPU 峰值: 90.16%
- 内存: 1100.01MB → 1333.28MB, 增长 233.27MB

六、场景2测试结果

样本数：1500 错误率：0.0% 吞吐量：198.12 TPS

响应时间

均值0.71 / P95 2.0 / 最大36.0 ms

时间分解 (ms)

- 连接时间(Connect)：均值0.06 / P95 0.0 / 最大32.0 ms
- 网络/等待时间(Latency-Connect)：均值0.64 / P95 1.0 / 最大6.0 ms
- 下载时间(Elapsed-Latency)：均值0.01 / P95 0.0 / 最大1.0 ms

服务器资源

- CPU 峰值：92.86%
- 内存：1104.7MB → 1324.39MB, 增长 219.69MB

两场景对比

压力测试相比性能测试：吞吐量变化 781.3%，95% 响应时间变化 -33.3%。

七、瓶颈分类分析

场景1：整体性能良好，无明显硬件或架构瓶颈。

- [低] 网络连接：连接时间均值 0.1ms, Keep-Alive 生效，连接建立不是瓶颈。
- [低] 服务器处理：首字节等待均值 1.51ms, Spring Boot 接口处理较快。
- [高] CPU 资源：压测期间 JVM 进程 CPU 峰值 90.16%，CPU 可能成为瓶颈。
- [中] 内存泄漏风险：测试期间内存增长 233.27MB，需关注是否存在内存泄漏。
- [低] 功能/稳定性：错误率 0%，检查点（状态码、响应内容、响应时间）全部通过。

场景2：整体性能良好，无明显硬件或架构瓶颈。

- [低] 网络连接：连接时间均值 0.06ms, Keep-Alive 生效，连接建立不是瓶颈。
- [低] 服务器处理：首字节等待均值 0.64ms, Spring Boot 接口处理较快。
- [高] CPU 资源：压测期间 JVM 进程 CPU 峰值 92.86%，CPU 可能成为瓶颈。
- [中] 内存泄漏风险：测试期间内存增长 219.69MB，需关注是否存在内存泄漏。
- [低] 功能/稳定性：错误率 0%，检查点（状态码、响应内容、响应时间）全部通过。

八、实验结论

1. 脚本已增强：事务控制器封装业务、CSV 参数化用户名、三重检查点保障结果正确性。
2. 完成性能测试与压力测试两个场景，覆盖不同并发强度下的 TPS 与响应时间。
3.

通过	JMeter	时间分解与服务器
----	--------	----------

CPU/内存监测，对连接、网络等待、下载时间及资源占用进行瓶颈分类；本示例接口逻辑简单，两场景下均无功能性错误，主要延迟集中在首字节等待，CPU/内存未达饱和。
4. HTML 详细报告：[jmeter/results/scenario1-report/index.html](#) 与 [scenario2-report/index.html](#)